

```

import numpy as np
import pandas as pd
import re
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt

```

```

df = pd.read_csv("SMSSpamCollection (1)", sep='\t', header=None, names=['label', 'text'])

print(df.head())
print(df['label'].value_counts())

```

```

label      text
0  ham  Go until jurong point, crazy.. Available only ...
1  ham                Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3  ham  U dun say so early hor... U c already then say...
4  ham  Nah I don't think he goes to usf, he lives aro...
label
ham      4825
spam      747
Name: count, dtype: int64

```

```

df['label'] = df['label'].map({'ham':0, 'spam':1})

```

```

def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-zA-Z\s', '', text)
    return text

```

```

df['text'] = df['text'].apply(clean_text)

```

```

tokenized_text = [text.split() for text in df['text']]

```

```

MAX_LEN = 50

def pad_sequence(seq, max_len):
    if len(seq) < max_len:
        seq += ['<PAD>'] * (max_len - len(seq))
    else:
        seq = seq[:max_len]

```

```

return seq

tokenized_text = [pad_sequence(seq, MAX_LEN) for seq in tokenized_text]

```

```

vocab = {'<PAD>':0}
for sentence in tokenized_text:
    for word in sentence:
        if word not in vocab:
            vocab[word] = len(vocab)

```

```

!rm -f glove.6B.zip
!wget http://nlp.stanford.edu/data/glove.6B.zip
!unzip -o glove.6B.zip glove.6B.100d.txt

embedding_dim = 100
embeddings_index = {}

with open("glove.6B.100d.txt", encoding="utf8") as f:
    for line in f:
        values = line.split()
        word = values[0]
        vector = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = vector

```

```

--2026-02-19 04:31:48-- http://nlp.stanford.edu/data/glove.6B.zip
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... connect
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
--2026-02-19 04:31:49-- https://nlp.stanford.edu/data/glove.6B.zip
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connect
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2026-02-19 04:31:49-- https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
Saving to: 'glove.6B.zip'

glove.6B.zip      100%[=====>] 822.24M  4.97MB/s   in 2m 43s

2026-02-19 04:34:32 (5.05 MB/s) - 'glove.6B.zip' saved [862182613/862182613]

Archive:  glove.6B.zip
  inflating: glove.6B.100d.txt

```

```

embedding_matrix = np.zeros((len(vocab), embedding_dim))

for word, idx in vocab.items():
    if word in embeddings_index:

```

```
embedding_matrix[idx] = embeddings_index[word]
```

```
embedding_matrix = torch.tensor(embedding_matrix, dtype=torch.float32)
```

```
numerical_data = [[vocab[word] for word in sentence] for sentence in tokeniz  
labels = df['label'].values
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    numerical_data, labels, test_size=0.2, random_state=42  
)
```

```
class TextCNN(nn.Module):  
    def __init__(self, vocab_size, embedding_dim, embedding_matrix):  
        super(TextCNN, self).__init__()  
  
        self.embedding = nn.Embedding.from_pretrained(  
            embedding_matrix, freeze=False  
        )  
  
        self.conv1 = nn.Conv1d(embedding_dim, 128, kernel_size=5)  
        self.pool = nn.MaxPool1d(2)  
        self.fc1 = nn.Linear(128 * 23, 64)  
        self.fc2 = nn.Linear(64, 1)  
        self.dropout = nn.Dropout(0.5)  
  
    def forward(self, x):  
        x = self.embedding(x)  
        x = x.permute(0, 2, 1)  
        x = torch.relu(self.conv1(x))  
        x = self.pool(x)  
        x = x.view(x.size(0), -1)  
        x = self.dropout(torch.relu(self.fc1(x)))  
        x = torch.sigmoid(self.fc2(x))  
        return x
```

```
train_dataset = SMSDataset(X_train, y_train)  
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)  
  
model = TextCNN(len(vocab), embedding_dim, embedding_matrix).to(device)  
criterion = nn.BCELoss()  
optimizer = optim.Adam(model.parameters(), lr=0.001)  
  
EPOCHS = 15  
  
for epoch in range(EPOCHS):  
    model.train()  
    epoch_loss = 0  
    for batch_X, batch_y in train_loader:
```

```

batch_X, batch_y = batch_X.to(device), batch_y.to(device)

optimizer.zero_grad()
outputs = model(batch_X)
loss = criterion(outputs, batch_y)
loss.backward()
optimizer.step()
epoch_loss += loss.item()

if (epoch + 1) % 5 == 0:
    print(f"Epoch {epoch+1}, Avg Loss: {epoch_loss/len(train_loader)}")

```

Epoch 5, Avg Loss: 0.009224583558437058
 Epoch 10, Avg Loss: 0.0032954694627339415
 Epoch 15, Avg Loss: 0.003618467309361451

```

from sklearn.metrics import classification_report

# Generate the report as a dictionary
report_dict = classification_report(all_labels, all_preds, target_names=['ha

# Convert to DataFrame for a nice table display
report_df = pd.DataFrame(report_dict).transpose()

print("Classification Report Table:")
display(report_df)

```

Classification Report Table:

	precision	recall	f1-score	support	
ham	0.989712	0.995859	0.992776	966.000000	
spam	0.972028	0.932886	0.952055	149.000000	
accuracy	0.987444	0.987444	0.987444	0.987444	
macro avg	0.980870	0.964373	0.972415	1115.000000	
weighted avg	0.987349	0.987444	0.987334	1115.000000	

Next steps: [Generate code with report_df](#) [New interactive sheet](#)

```

# ===== FINAL EVALUATION STEP =====

model.eval()

all_preds = []
all_labels = []

with torch.no_grad():
    for inputs, labels in test_loader:
        inputs = inputs.to(device)
        labels = labels.to(device)

        outputs = model(inputs)

```

```
# Note: Model forward already has sigmoid, outputs are probabilities
preds = (outputs > 0.5).int()

all_preds.extend(preds.cpu().numpy())
all_labels.extend(labels.cpu().numpy())

# Convert to numpy arrays
all_preds = np.array(all_preds).flatten()
all_labels = np.array(all_labels).flatten()

# Metrics
accuracy = accuracy_score(all_labels, all_preds)
precision = precision_score(all_labels, all_preds)
recall = recall_score(all_labels, all_preds)
f1 = f1_score(all_labels, all_preds)
cm = confusion_matrix(all_labels, all_preds)

print("Accuracy: {:.2f}%".format(accuracy * 100))
print("Precision: {:.2f}%".format(precision * 100))
print("Recall: {:.2f}%".format(recall * 100))
print("F1 Score: {:.2f}%".format(f1 * 100))
print("Confusion Matrix:\n", cm)
```

```
Accuracy: 98.74%
Precision: 97.20%
Recall: 93.29%
F1 Score: 95.21%
Confusion Matrix:
[[962   4]
 [ 10 139]]
```